CYBERSTER

Blue Team Internship

WEEK 1 REPORT

SOC Foundations and Lab Setup

Program	Cyberster Blue Team Internship
Phase	Phase One — SOC Operations and SIEM Foundations
Track	Blue Team
Week	Week 1 — Days 1 through 7
Submitted By	Ahmad Hassan
Roll no	CSI-B1-349
Submission Date	7/3/2026
Instructor	Abdullah Zia

TABLE OF CONTENTS

1. Overview 4
   1.1 Lab Environment Summary 4
2. Day 1 — Networking Fundamentals for SOC 5
   2.1 OSI Model and TCP/IP Model 5
   2.2 Subnetting and CIDR Notation 5
   2.3 SOC-Relevant Ports and Protocols 5
   2.4 Protocol Analysis 6
      ARP — Address Resolution Protocol 6
      DNS — Domain Name System 6
      HTTP and HTTPS 6
      ICMP — Internet Control Message Protocol 6
   2.5 Wireshark Live Capture 6
   2.6 Day 1 Deliverables 9
3. Day 2 — VMware Lab Setup 10
   3.1 Lab Topology 10
   3.2 Virtual Machine Configuration 10
      Ubuntu 22.04 — Wazuh Server 10
      Kali Linux 2024.3 — Attacker Machine 11
      Windows 10 Pro — Monitored Endpoint 12
   3.3 Connectivity Tests 13
   3.4 Snapshots ..... **Error! Bookmark not defined.**
   3.5 Problems Encountered 14
   3.6 What I Understood 14
4. Days 3–4 — Wazuh Manager Installation 15
   4.1 Wazuh Architecture 15
   4.2 Static IP Assignment 15
   4.3 Installation Process 15
   4.4 Service Verification 16
   4.5 Firewall Configuration 17
   4.6 Dashboard Access 17
   4.7 What I Understood 17
5. Days 5–6 — Wazuh Agent Deployment 18
   5.1 Deployment Method 18
   5.2 Kali Linux Agent Installation 18

5.3 Windows 10 Agent Installation 19
5.4 Both Agents Active in Dashboard 20
5.5 ossec.conf Review 20
5.6 Pipeline Verification — Test Alerts 21
5.7 What I Understood 21
6. Day 7 — Dashboard Navigation and Alert Triage 22
6.1 Dashboard Modules Explored 22
6.2 Wazuh Rule Levels 23
6.3 Threat Hunting — Queries Practiced 24
6.4 MITRE ATT&CK Mapping 24
6.5 Alert Triage Report — Top 10 Alert Types 24
6.6 What I Understood 25
7. Week 1 Reflections 25
7.1 Key Technical Learnings 25
7.2 Problems Encountered and Resolved 25
7.3 What Comes Next 26

1. Overview

This report documents the work completed during Week 1 of the Cyberster Blue Team Internship. The objective of this week was to build a professional defensive lab environment from scratch and develop a practical understanding of network traffic analysis and SIEM architecture.

The week was structured across seven days, beginning with networking fundamentals and concluding with alert triage on the Wazuh SIEM dashboard. All lab work was performed inside VMware Workstation using three virtual machines.

1.1 Lab Environment Summary

OS	Role	IP Address	RAM	CPU	Storage
Ubuntu 22.04	Wazuh SIEM Server	192.168.81.135	8 GB	2	50 GB
Kali Linux 2024.3	Attacker Machine / Agent	192.168.81.136	4 GB	2	50 GB
Windows 10 Pro	Monitored Endpoint / Agent	192.168.81.129	4 GB	2	50 GB

2. Day 1 — Networking Fundamentals for SOC

Before any virtual machine was configured, I reviewed the core networking concepts that underpin SOC work. This section documents what I studied and what I understood from each topic.

2.1 OSI Model and TCP/IP Model

The OSI model divides network communication into seven layers, each with a specific responsibility. The TCP/IP model condenses this into four layers. Understanding both models matters in SOC work because alerts reference specific layers — a port scan operates at Layer 4, ARP spoofing at Layer 2, and DNS tunnelling at Layer 7.

To make this concrete, I traced a web request from browser to server:

- Browser forms an HTTP GET request (Application Layer 7)
- TLS encryption is applied for HTTPS (Presentation Layer 6)
- TCP connection is established via 3-way handshake on port 443 (Transport Layer 4)
- IP packet is addressed and routed toward the destination (Network Layer 3)
- MAC address resolution via ARP happens on the local network (Data Link Layer 2)
- Bits are transmitted over the physical medium (Physical Layer 1)

2.2 Subnetting and CIDR Notation

A subnet mask defines which portion of an IP address identifies the network and which identifies the host. CIDR notation expresses this as a prefix length. For example, 192.168.56.0/24 means the first 24 bits are the network — leaving 8 bits for hosts, giving 254 usable addresses.

Practical rule: two IP addresses can communicate directly without a router if their network portions are identical. 192.168.56.10 and 192.168.56.20 on a /24 subnet are on the same network and communicate directly. 192.168.56.10 and 192.168.57.10 are on different networks and require routing.

2.3 SOC-Relevant Ports and Protocols

Port	Protocol	Purpose	Suspicious Indicator
22	**SSH**	Remote server login	Failed login flood = brute force
53	**DNS**	Resolves domain names to IPs	High query rate = DNS tunneling
80	**HTTP**	Unencrypted web traffic	Credentials sent in plain text
443	**HTTPS**	Encrypted web traffic	Unexpected self-signed cert = MITM
25	**SMTP**	Sends email	Bulk outbound = spam / phishing
445	**SMB**	Windows file sharing	Primary vector for ransomware (WannaCry)
3389	**RDP**	Windows remote desktop	Exposed to internet = critical risk
8080	**HTTP Alt**	Alternative web port	C2 (Command & Control) beaconing

2.4 Protocol Analysis

ARP — Address Resolution Protocol

ARP resolves IP addresses to MAC addresses on a local network segment. A normal ARP exchange is a broadcast asking who owns a given IP, followed by a unicast reply. Suspicious ARP traffic includes gratuitous ARP replies that are not responses to requests — a technique used in ARP spoofing to redirect traffic through an attacker's machine.

DNS — Domain Name System

DNS translates human-readable domain names into IP addresses. A normal exchange is a short query and response, usually under 512 bytes. Suspicious DNS traffic includes unusually long domain names, high-frequency queries to a single domain, or TXT record responses carrying encoded data — signs of DNS tunnelling.

HTTP and HTTPS

HTTP carries web content in plain text. HTTPS adds TLS encryption. Suspicious HTTP includes POST requests containing credentials in the body, or traffic to known malicious domains. Suspicious HTTPS includes self-signed certificates on domains that should carry trusted certificates, or traffic to IP addresses rather than domain names.

ICMP — Internet Control Message Protocol

ICMP is used for network diagnostics, most commonly ping. Normal ICMP is occasional and small. Suspicious ICMP includes a high volume of echo requests (ICMP flood used in DDoS), or ICMP packets with unusually large payloads that may carry encoded data (ICMP tunnelling).

2.5 Wireshark Live Capture

I opened Wireshark on the host machine and captured live traffic for ten minutes while browsing normally. I identified the following five protocols in the capture:

- DNS — Queries resolving domain names before HTTP connections. Visible as short request-response pairs on UDP port 53.
- TCP — Three-way handshakes (SYN, SYN-ACK, ACK) establishing connections before data transfer. Visible on ports 80 and 443.
- HTTP — Unencrypted web requests on port 80. GET and POST methods visible in plain text in the packet detail pane.
- TLS (HTTPS) — Encrypted traffic on port 443. The handshake messages (Client Hello, Server Hello) are visible but payload is encrypted.
- ARP — Broadcast requests resolving gateway IP to MAC address. Short exchanges visible at start of each new connection.

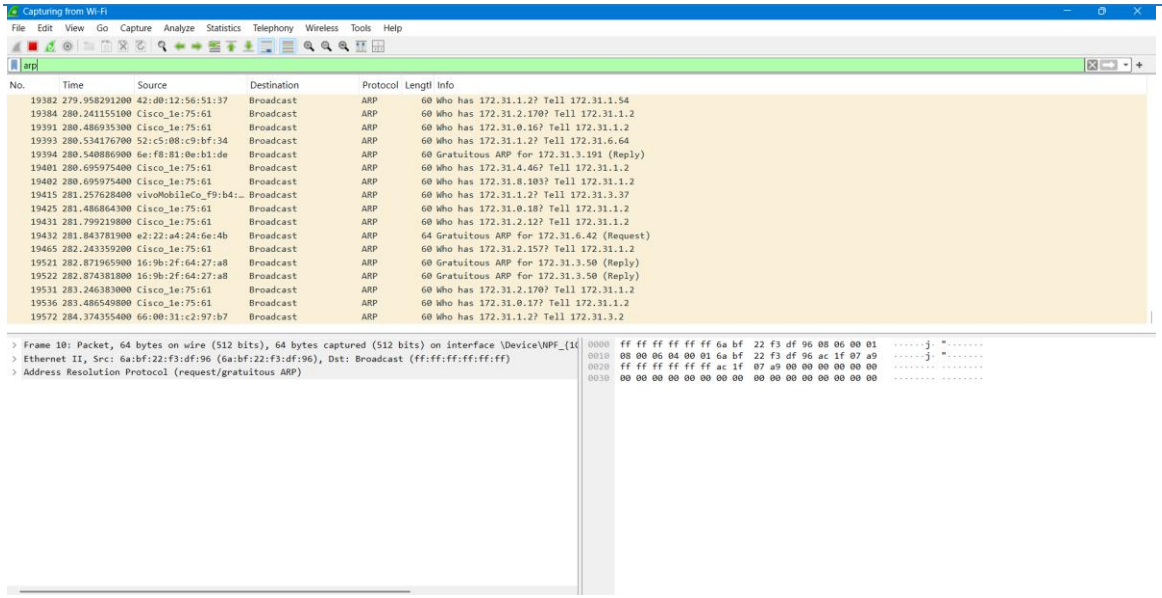


Figure 1 Arp

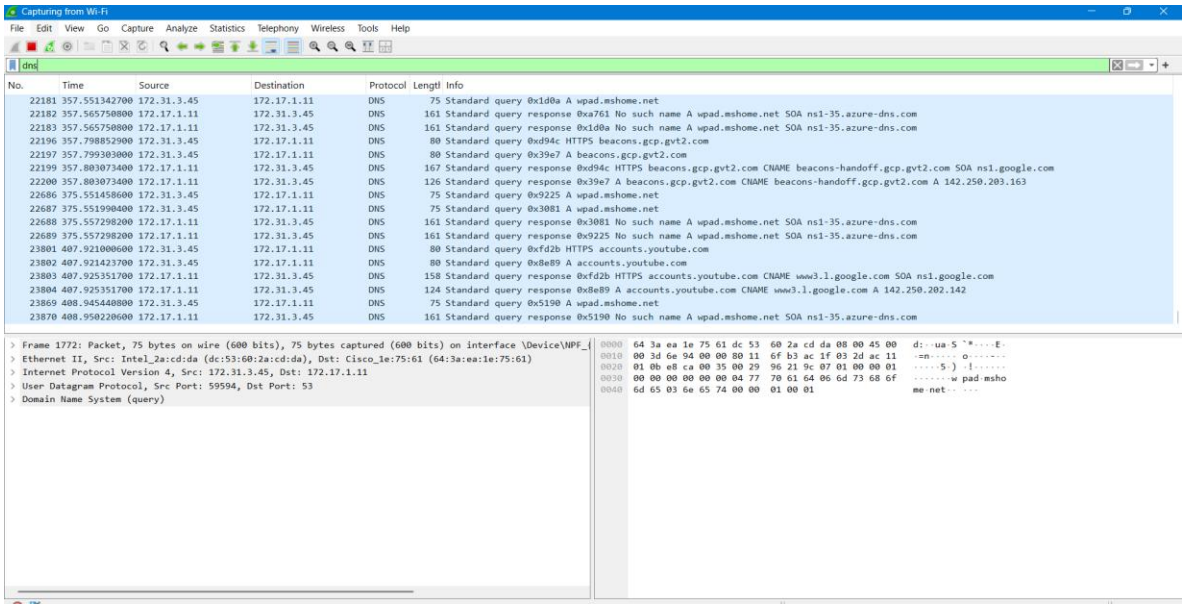


Figure 2 Dns:

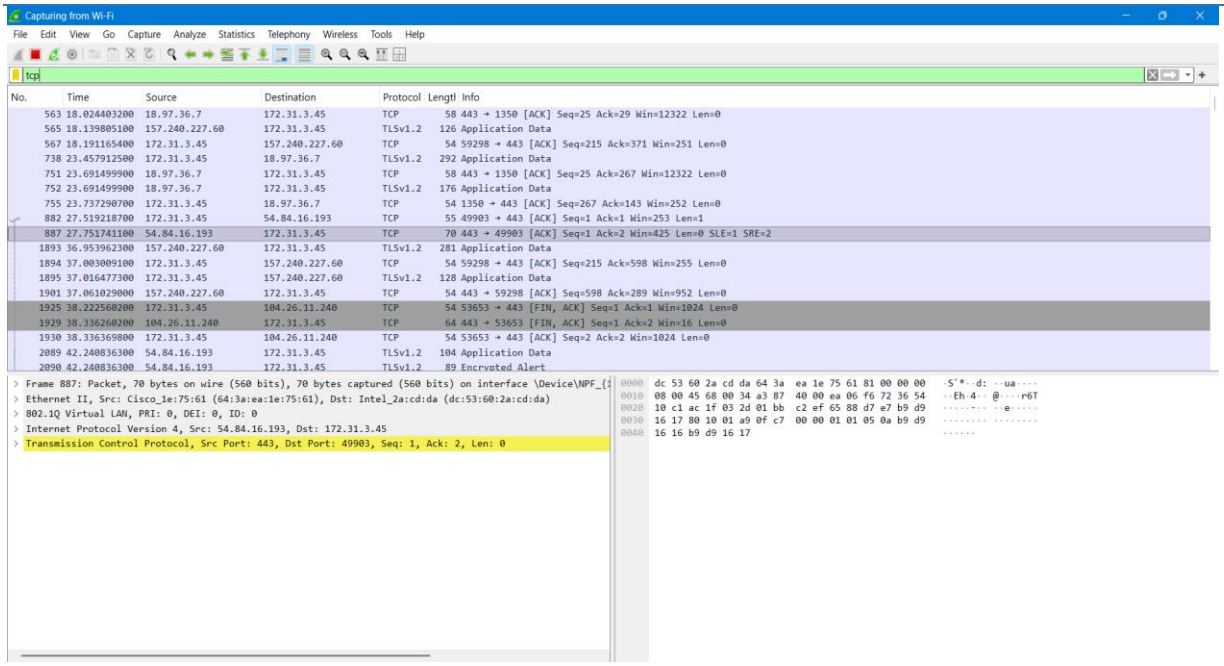


Figure 3 Tcp:

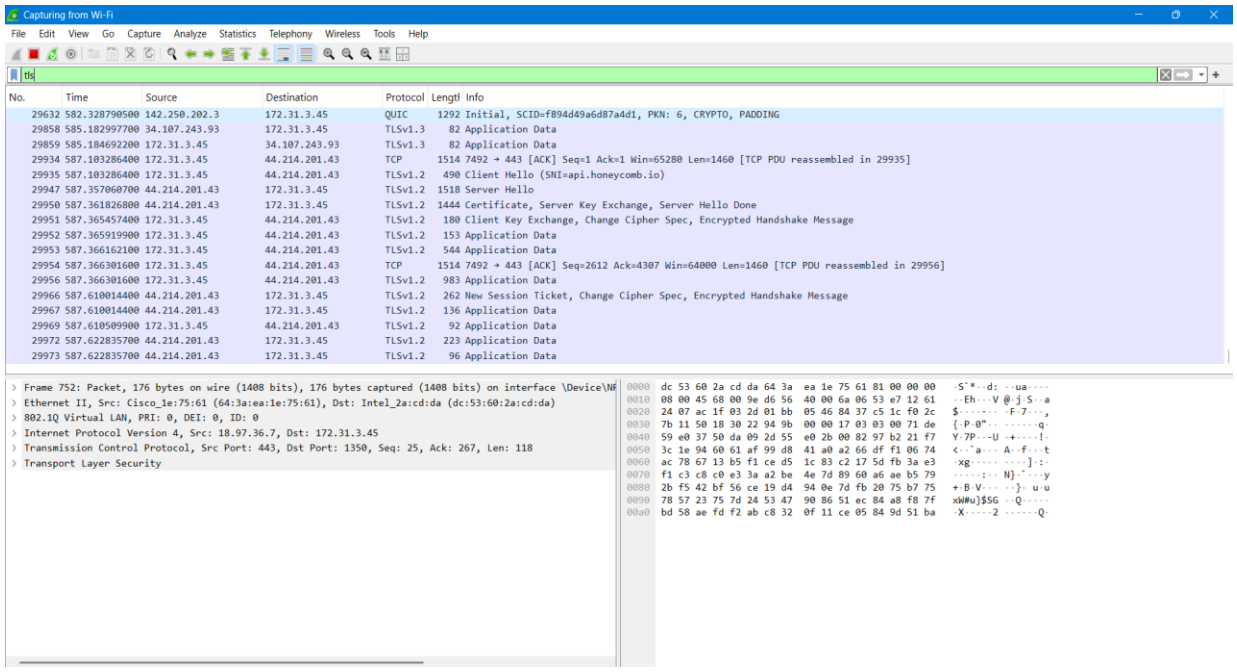


Figure 4 Tls:

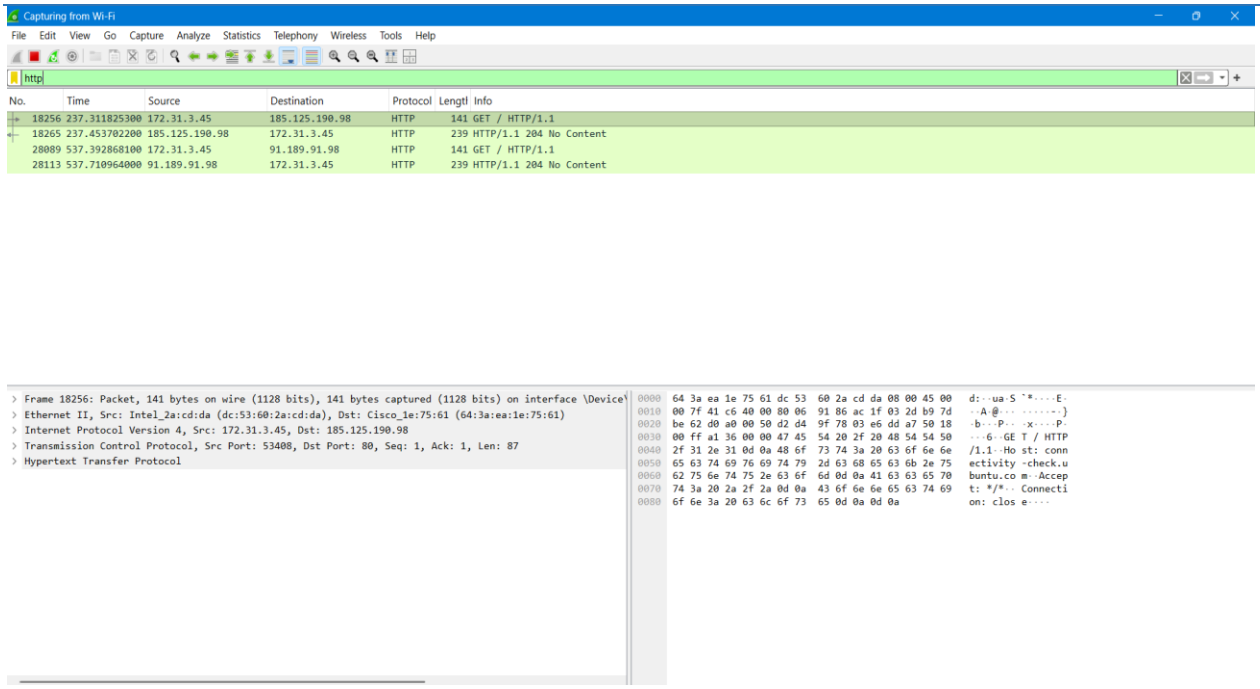


Figure 5 Http:

2.6 Day 1 Deliverables

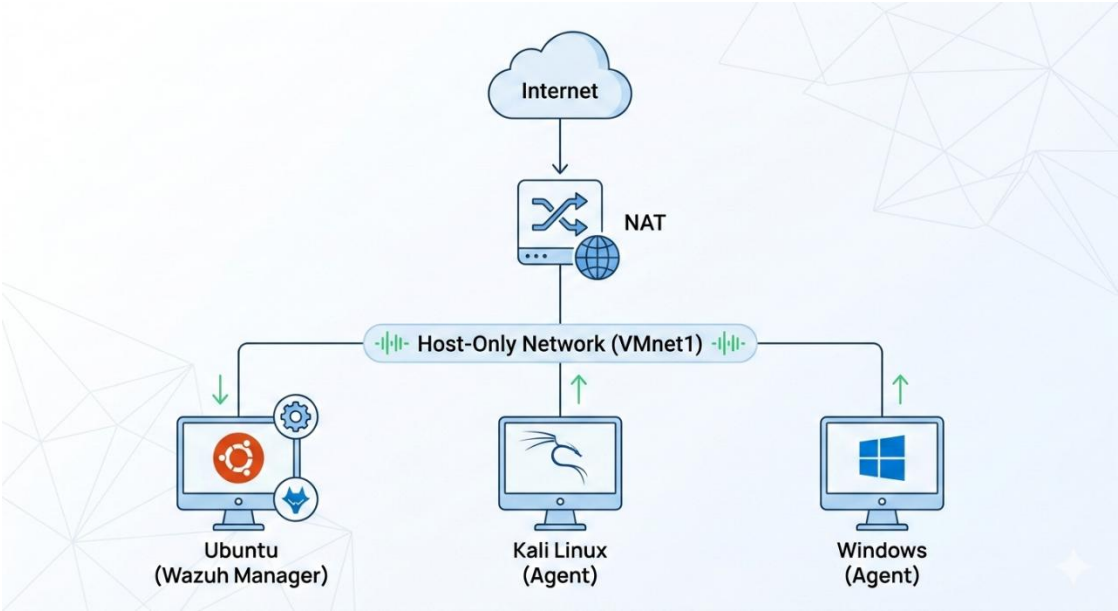
- Network topology diagram — see Section 3.1
- Wireshark capture file with five protocols identified — screenshot above
- Written protocol summary — documented in Sections 2.4 and 2.5

3. Day 2 — VMware Lab Setup

This day involved building the three-machine lab environment that is used throughout Phase One. VMware Workstation was used instead of VirtualBox, with equivalent networking modes configured.

3.1 Lab Topology

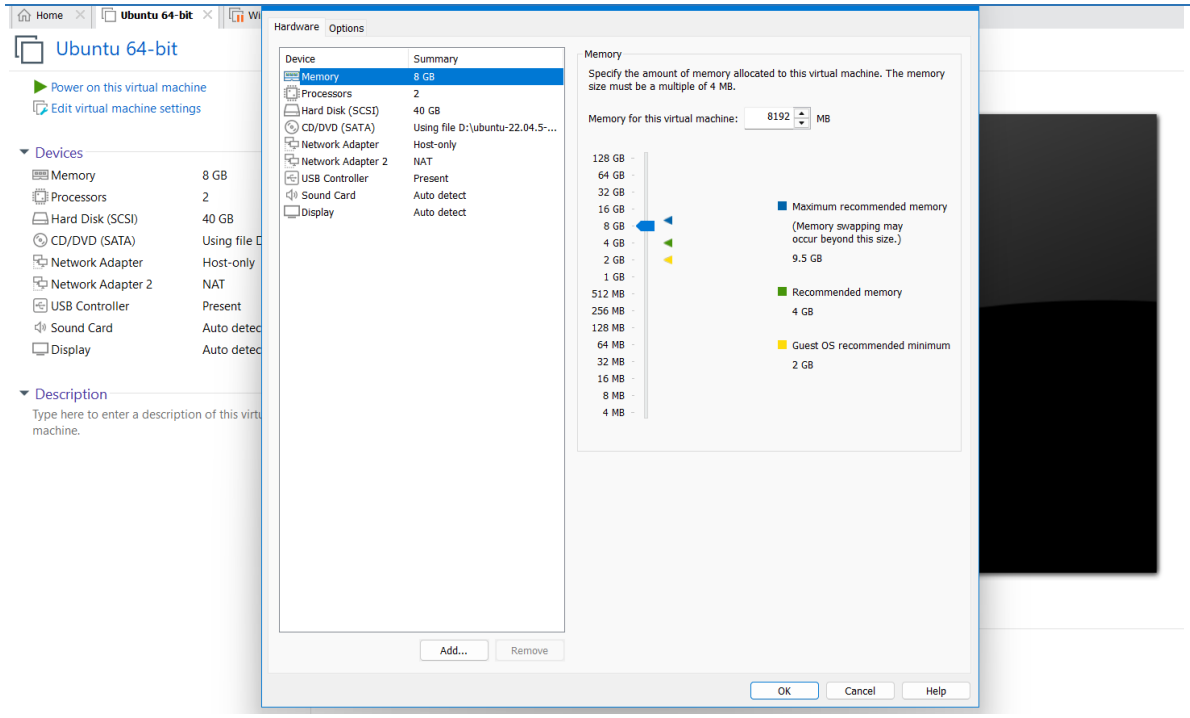
Each virtual machine was configured with two network adapters. The first adapter connects to the Host-Only network (VMnet1) so the three machines can communicate with each other and with the host. The second adapter connects to NAT so each machine can reach the internet for package downloads and updates.



3.2 Virtual Machine Configuration

Ubuntu 22.04 — Wazuh Server

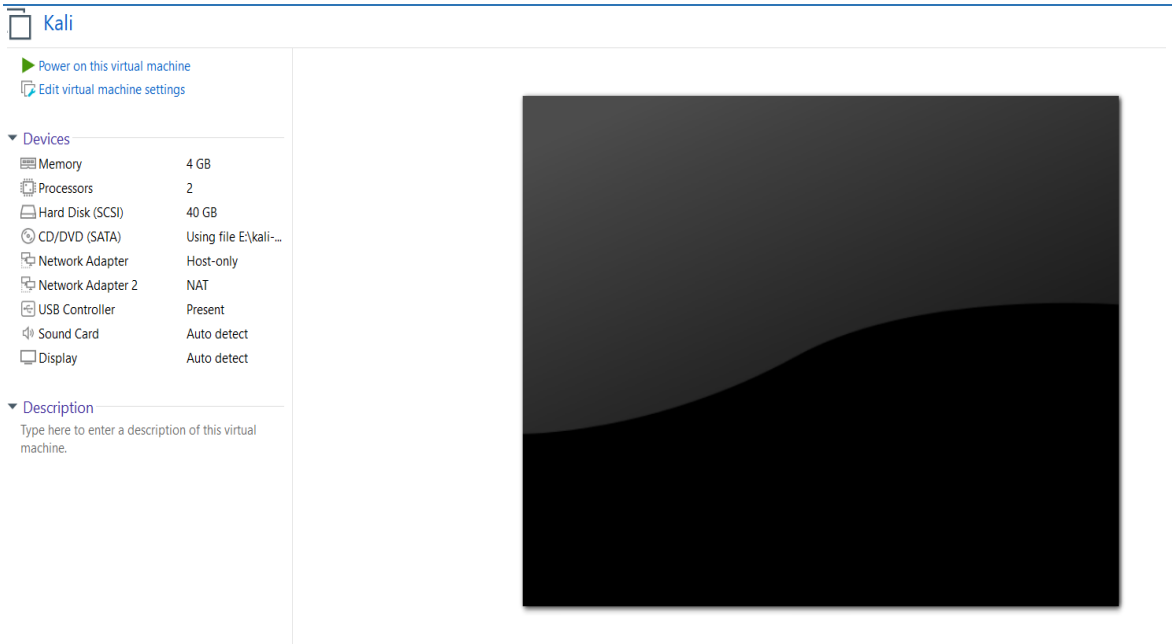
- RAM: 4 GB — required for Wazuh Indexer
- CPU: 2 cores
- Storage: 40 GB
- Adapter 1: Host-Only (VMnet1) — IP: 192.168.81.135
- Adapter 2: NAT — internet access




```
ahmad66182@ahmad66182-virtual-machine:~/Desktop$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:76:fd:13 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.136.134/24 brd 192.168.136.255 scope global dynamic noprefixroute ens33
       valid_lft 1700sec preferred_lft 1700sec
    inet6 fe80::578c:277d:1ef9:7b64/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:76:fd:1d brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 192.168.81.137/24 brd 192.168.81.255 scope global dynamic noprefixroute ens37
       valid_lft 1700sec preferred_lft 1700sec
    inet6 fe80::cd20:52c3:5450:e1c6/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Kali Linux 2024.3 — Attacker Machine

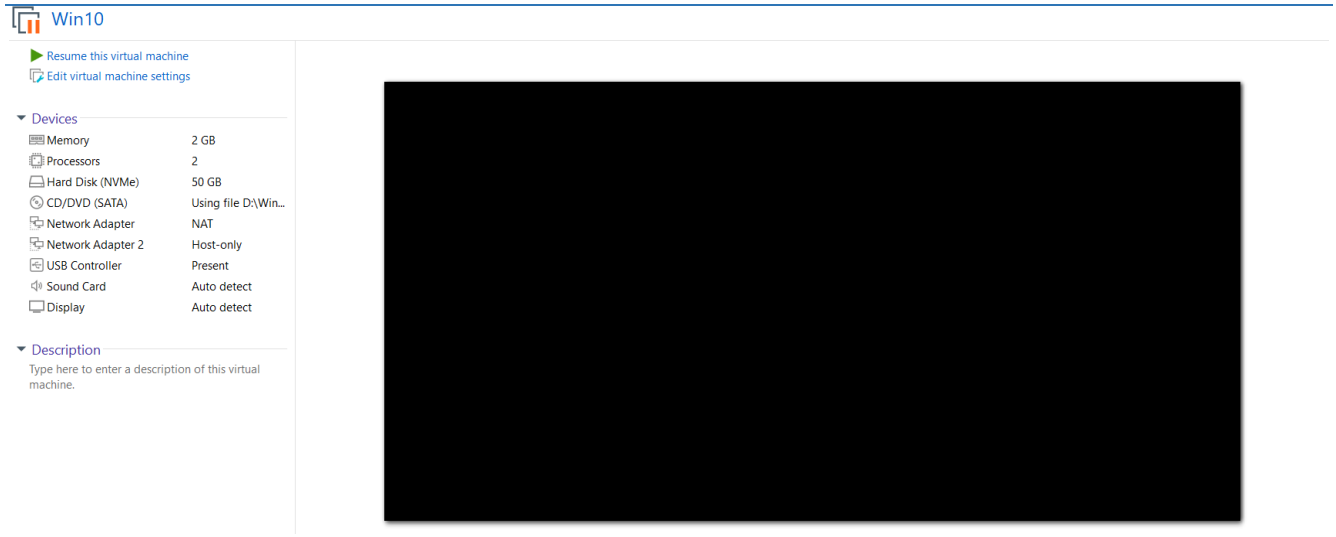
- RAM: 4 GB
- CPU: 2 cores
- Storage: 50 GB
- Adapter 1: Host-Only — IP: 192.168.81.136
- Adapter 2: NAT — internet access




```
File  Actions  Edit  View  Help

┌──(mrkali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:a5:e7:5d brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:a5:e7:67 brd ff:ff:ff:ff:ff:ff
    inet 192.168.81.138/24 brd 192.168.81.255 scope global dynamic noprefixro
ute eth1
       valid_lft 1740sec preferred_lft 1740sec
    inet6 fe80::20c:29ff:fea5:e767/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Windows 10 Pro — Monitored Endpoint

- RAM: 4 GB
- CPU: 2 cores
- Storage: 50 GB
- Adapter 1: Host-Only — IP: 192.168.81.129
- Adapter 2: NAT — internet access



C:\Windows\system32\cmd.exe

```
Wireless LAN adapter Local Area Connection* 2:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Ethernet adapter VMware Network Adapter VMnet1:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::c1c4:529b:ae12:e7f7%15
   IPv4 Address. . . . . . . . . . . : 192.168.136.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Ethernet adapter VMware Network Adapter VMnet8:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::3fd:1b11:fbe7:1cb7%8
   IPv4 Address. . . . . . . . . . . : 192.168.81.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . : iub.edu.pk
   Link-local IPv6 Address . . . . . : fe80::604d:e5e9:b4a7:5d92%5
   IPv4 Address. . . . . . . . . . . : 172.31.3.45
   Subnet Mask . . . . . . . . . . . : 255.255.0.0
   Default Gateway . . . . . . . . . : 172.31.1.2

C:\Users\user>
```

3.3 Connectivity Tests

After all three VMs were configured and booted, I ran ping tests to confirm that the Host-Only network was functioning correctly and that all machines could communicate with each other.

- Kali → Ubuntu (192.168.81.137): Success — 0% packet loss
- Windows → Ubuntu (192.168.81.137): Success — 0% packet loss
- Ubuntu → Kali (192.168.81.138): Success
- All three VMs → 8.8.8.8 (internet): Success

```
└─$ ping 192.168.81.137
PING 192.168.81.137 (192.168.81.137) 56(84) bytes of data.
64 bytes from 192.168.81.137: icmp_seq=1 ttl=64 time=1.48 ms
64 bytes from 192.168.81.137: icmp_seq=2 ttl=64 time=2.74 ms
64 bytes from 192.168.81.137: icmp_seq=3 ttl=64 time=1.49 ms
64 bytes from 192.168.81.137: icmp_seq=4 ttl=64 time=3.14 ms
64 bytes from 192.168.81.137: icmp_seq=5 ttl=64 time=1.62 ms
64 bytes from 192.168.81.137: icmp_seq=6 ttl=64 time=1.67 ms
^C
── 192.168.81.137 ping statistics ──
6 packets transmitted, 6 received, 0% packet loss, time 5008ms
rtt min/avg/max/mdev = 1.478/2.021/3.137/0.661 ms
```

-

```
82@anmad66182-virtual-machine:~/Desktop$ ping 192.168.81.138
.168.81.138 (192.168.81.138) 56(84) bytes of data.
 from 192.168.81.138: icmp_seq=1 ttl=64 time=4.82 ms
 from 192.168.81.138: icmp_seq=2 ttl=64 time=2.87 ms
 from 192.168.81.138: icmp_seq=3 ttl=64 time=1.20 ms
 from 192.168.81.138: icmp_seq=4 ttl=64 time=2.55 ms
 from 192.168.81.138: icmp_seq=5 ttl=64 time=3.50 ms
 from 192.168.81.138: icmp_seq=6 ttl=64 time=3.75 ms
 from 192.168.81.138: icmp_seq=7 ttl=64 time=2.38 ms
 from 192.168.81.138: icmp_seq=8 ttl=64 time=3.69 ms

168.81.138 ping statistics ---
s transmitted, 8 received, 0% packet loss, time 7013ms
avg/max/mdev = 1.195/3.093/4.824/1.024 ms
```

```
C:\Users\user>ping 192.168.81.137

Pinging 192.168.81.137 with 32 bytes of data:
Reply from 192.168.81.137: bytes=32 time=2ms TTL=64
Reply from 192.168.81.137: bytes=32 time=2ms TTL=64
Reply from 192.168.81.137: bytes=32 time=1ms TTL=64
Reply from 192.168.81.137: bytes=32 time=5ms TTL=64

Ping statistics for 192.168.81.137:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 5ms, Average = 2ms
```

3.5 Problems Encountered

Windows 10 VM initially had no internet access. The second network adapter was set to LAN Segment instead of NAT. After correcting this in VMware settings and restarting the VM, internet connectivity was restored. The lesson here is to always verify adapter settings from inside the VM using ipconfig or ip a, not just from the VMware settings panel.

3.6 What I Understood

The lab runs in an isolated environment so that attack simulation activity — such as SSH brute force from Kali — does not affect the real network. The Host-Only network provides that isolation. The NAT adapter allows internet access without exposing the VMs directly to the network. These two adapters serve different purposes and both are necessary.

4. Days 3–4 — Wazuh Manager Installation

Wazuh is the SIEM platform used throughout Phase One. SIEM stands for Security Information and Event Management. Its job is to collect log data from endpoints, apply detection rules, and surface events that require investigation. Wazuh consists of three components that all run on the Ubuntu Server VM.

4.1 Wazuh Architecture

- Wazuh Manager: Receives log data from agents, applies detection rules, and generates alerts.
- Wazuh Indexer: Stores all log data and alert records in a searchable format using OpenSearch.
- Wazuh Dashboard: The web interface where analysts view, filter, and investigate alerts.

All three components must be running before agents can connect. The Indexer is the most memory-intensive component — this is why 8 GB RAM was allocated to the Ubuntu VM.

4.2 Static IP Assignment

Before installation, I assigned a static IP to the Host-Only interface on Ubuntu Server. This is critical because agents will point to this IP address. If it changes after agents are configured, they lose their connection and must be reconfigured.

- File edited: /etc/netplan/00-installer-config.yaml
- Host-Only adapter (ens33): static IP 192.168.81.135/24
- NAT adapter (ens34): DHCP
- Command run: sudo netplan apply

```
ahmad66182@ahmad66182-virtual-machine:~/Desktop$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:76:fd:13 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.81.135/24 brd 192.168.81.255 scope global dynamic noprefixroute ens33
       valid_lft 1716sec preferred_lft 1716sec
    inet6 fe80::578c:277d:1ef9:7b64/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

4.3 Installation Process

I followed the official Wazuh quickstart documentation. I did not use any third-party tutorial, as the task brief explicitly warns that these are frequently outdated. The quickstart script installs and configures all three components in a single run.

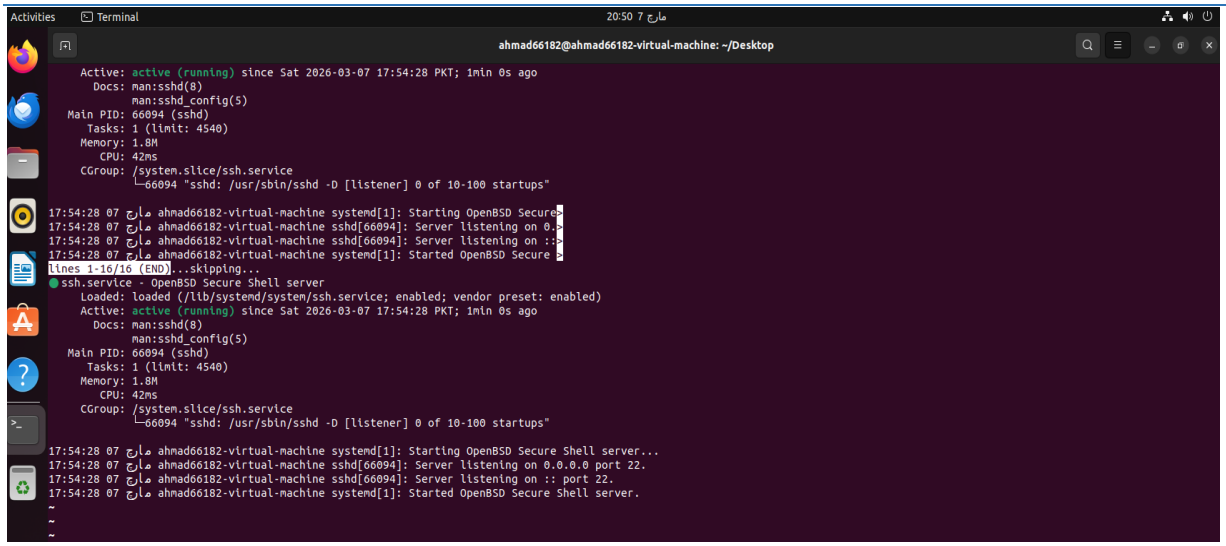
- Downloaded install script: curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
- Configured config.yml with Ubuntu Server IP: 192.168.81.135
- Ran installation: sudo bash wazuh-install.sh -a
- Installation duration: approximately 25 minutes
- Admin credentials were generated at end of installation and saved immediately

```
ahmad66182@ahmad66182-virtual-machine:~/Desktop$ cd ~/Desktop
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
07/03/2026 15:23:49 INFO: Starting Wazuh installation assistant. Wazuh version: 4.14.3
07/03/2026 15:23:49 INFO: Verbose logging redirected to /var/log/wazuh-install.log
07/03/2026 15:24:01 INFO: --- Dependencies ----
07/03/2026 15:24:01 INFO: Installing gawk.
07/03/2026 15:24:14 INFO: Verifying that your system meets the recommended minimum hardware requirements.
07/03/2026 15:24:14 INFO: Wazuh web interface port will be 443.
07/03/2026 15:24:30 INFO: --- Dependencies ----
07/03/2026 15:24:30 INFO: Installing apt-transport-https.
07/03/2026 15:24:36 INFO: Installing debhelper.
07/03/2026 15:27:21 INFO: Wazuh repository added.
07/03/2026 15:27:21 INFO: --- Configuration files ---
07/03/2026 15:27:21 INFO: Generating configuration files.
07/03/2026 15:27:23 INFO: Generating the root certificate.
07/03/2026 15:27:23 INFO: Generating Admin certificates.
07/03/2026 15:27:23 INFO: Generating Wazuh indexer certificates.
07/03/2026 15:27:23 INFO: Generating Filebeat certificates.
07/03/2026 15:27:24 INFO: Generating Wazuh dashboard certificates.
07/03/2026 15:27:25 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, and passwords necessary for installation.
07/03/2026 15:27:26 INFO: --- Wazuh indexer ---
07/03/2026 15:27:26 INFO: Starting Wazuh indexer installation.
07/03/2026 15:34:07 INFO: Wazuh indexer installation finished.
07/03/2026 15:34:07 INFO: Wazuh indexer post-install configuration finished.
07/03/2026 15:34:07 INFO: Starting service wazuh-indexer.
07/03/2026 15:34:38 INFO: wazuh-indexer service started.
07/03/2026 15:34:38 INFO: Initializing Wazuh indexer cluster security settings.
07/03/2026 15:34:45 INFO: Wazuh indexer cluster security configuration initialized.
07/03/2026 15:34:45 INFO: Wazuh indexer cluster initialized.
07/03/2026 15:34:45 INFO: --- Wazuh server ---
07/03/2026 15:34:45 INFO: Starting the Wazuh manager installation.
07/03/2026 15:42:11 INFO: Wazuh manager installation finished.
07/03/2026 15:42:12 INFO: Wazuh manager vulnerability detection configuration finished.
07/03/2026 15:42:12 INFO: Starting service wazuh-manager.
07/03/2026 15:42:34 INFO: wazuh-manager service started.
07/03/2026 15:42:34 INFO: Starting Filebeat installation.
07/03/2026 15:43:21 INFO: Filebeat installation finished.
07/03/2026 15:43:24 INFO: Filebeat post-install configuration finished.
07/03/2026 15:43:24 INFO: Starting service filebeat.
07/03/2026 15:43:28 INFO: filebeat service started.
```

```
07/03/2026 15:43:28 INFO: Starting Wazuh dashboard installation.
07/03/2026 15:48:09 INFO: Wazuh dashboard installation finished.
07/03/2026 15:48:09 INFO: Wazuh dashboard post-install configuration finished.
07/03/2026 15:48:09 INFO: Starting service wazuh-dashboard.
07/03/2026 15:48:13 INFO: wazuh-dashboard service started.
07/03/2026 15:48:27 INFO: Updating the internal users.
07/03/2026 15:48:49 INFO: A backup of the internal users has been saved in the /etc/wazuh-indexer/internalusers-backup folder.
07/03/2026 15:49:20 INFO: The filebeat.yml file has been updated to use the Filebeat Keystore username and password.
07/03/2026 15:50:18 INFO: Initializing Wazuh dashboard web application.
07/03/2026 15:50:19 INFO: Wazuh dashboard web application not yet initialized. Waiting...
07/03/2026 15:50:35 INFO: Wazuh dashboard web application not yet initialized. Waiting...
07/03/2026 15:50:50 INFO: Wazuh dashboard web application initialized.
07/03/2026 15:50:50 INFO: --- Summary ---
07/03/2026 15:50:50 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
    User: admin
    Password: NpwJJ7xClS39jZD?HoQ.ipjeGbhXiBO4
07/03/2026 15:50:50 INFO: --- Dependencies ----
07/03/2026 15:50:50 INFO: Removing gawk.
07/03/2026 15:50:59 INFO: Installation finished.
```

4.4 Service Verification

After installation, I checked the status of all three services using systemctl. All three showed as active and running.



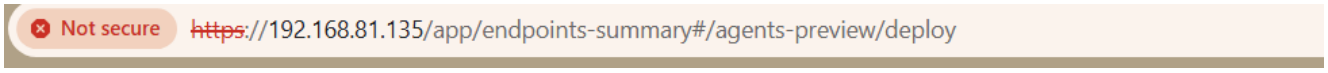
4.5 Firewall Configuration

I opened the required ports on Ubuntu Server's UFW firewall to allow agent communication:

- Port 443/tcp — Wazuh Dashboard (HTTPS)
- Port 1514/tcp — Agent log data transmission
- Port 1515/tcp — Agent registration

4.6 Dashboard Access

I accessed the Wazuh dashboard by navigating to https://192.168.81.135 in the browser. The browser displayed a certificate warning because Wazuh uses a self-signed certificate by default. I proceeded past the warning — this is expected and normal in a lab environment.



4.7 What I Understood

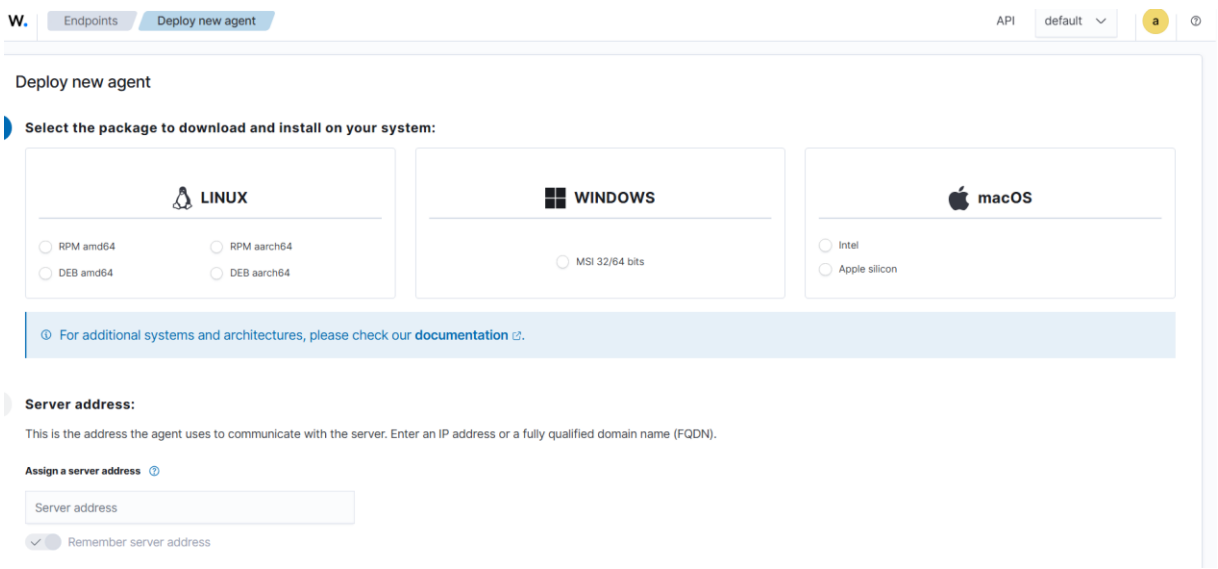
The three Wazuh components form a pipeline. Agents on endpoints send raw log data to the Manager. The Manager applies detection rules and decides which events are worth alerting on. The Indexer stores everything in a format that supports fast search queries. The Dashboard gives analysts the interface to work with all of it. This architecture is essentially the same in commercial SIEM platforms.

5. Days 5–6 — Wazuh Agent Deployment

With the Wazuh server running, I deployed agents on the two remaining machines: Kali Linux and Windows 10. An agent is a lightweight process that collects log data and system events from the endpoint and forwards them to the Wazuh Manager.

5.1 Deployment Method

I used the built-in deployment wizard in the Wazuh dashboard rather than manually writing installation commands. The wizard asks for the target operating system and the Manager IP address, then generates the correct installation command. This approach is safer than manual commands because it pre-fills the Manager IP and selects the correct package version.



5.2 Kali Linux Agent Installation

I ran the generated command in the Kali terminal. The command downloaded the agent package and installed it with the Manager IP pre-configured.

- Package downloaded: wazuh-agent_4.14.3-1_amd64.deb
- Manager IP configured: 192.168.81.137
- Agent name: KaliLinux

After installation, I ran the following commands to start and enable the agent service:

- sudo systemctl daemon-reload
- sudo systemctl enable wazuh-agent
- sudo systemctl start wazuh-agent

```
File  Actions  Edit  View  Help
● wazuh-agent.service - Wazuh agent
     Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)
     Active: reloading (reload) since Sat 2026-03-07 17:02:29 PKT; 23s ago
 Invocation: decdf6a04f1c4cf887a7a7ee53a02e97
    Process: 4508 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
   Cntrl PID: 5473 (wazuh-control)
      Tasks: 23 (limit: 4560)
     Memory: 89.8M (peak: 89.8M)
        CPU: 29.422s
     CGroup: /system.slice/wazuh-agent.service
             ├─4549 /var/ossec/bin/wazuh-agentd
             ├─5463 /bin/sh active-response/bin/restart.sh agent reload
             ├─5472 systemctl reload wazuh-agent
             ├─5473 /bin/sh /var/ossec/bin/wazuh-control reload
             ├─5615 /var/ossec/bin/wazuh-execd
             ├─5636 /var/ossec/bin/wazuh-syscheckd
             ├─5659 /var/ossec/bin/wazuh-logcollector
             └─5660 sleep 1

Mar 07 17:02:46 kali systemd[1]: Reloading wazuh-agent.service - Wazuh agent...
Mar 07 17:02:47 kali env[5473]: Killing wazuh-modulesd...
Mar 07 17:02:47 kali env[5473]: Killing wazuh-logcollector...
Mar 07 17:02:47 kali env[5473]: Killing wazuh-syscheckd...
Mar 07 17:02:47 kali env[5473]: Killing wazuh-execd...
Mar 07 17:02:49 kali env[5473]: Wazuh v4.14.3 Stopped
Mar 07 17:02:50 kali env[5473]: Starting Wazuh v4.14.3...
Mar 07 17:02:51 kali env[5473]: Started wazuh-execd...
Mar 07 17:02:51 kali env[5473]: wazuh-agentd already running...
Mar 07 17:02:52 kali env[5473]: Started wazuh-syscheckd...
~
~
~
~
~
~
~
~
~
~
~
~
~
lines 1-29/29 (END)
```

5.3 Windows 10 Agent Installation

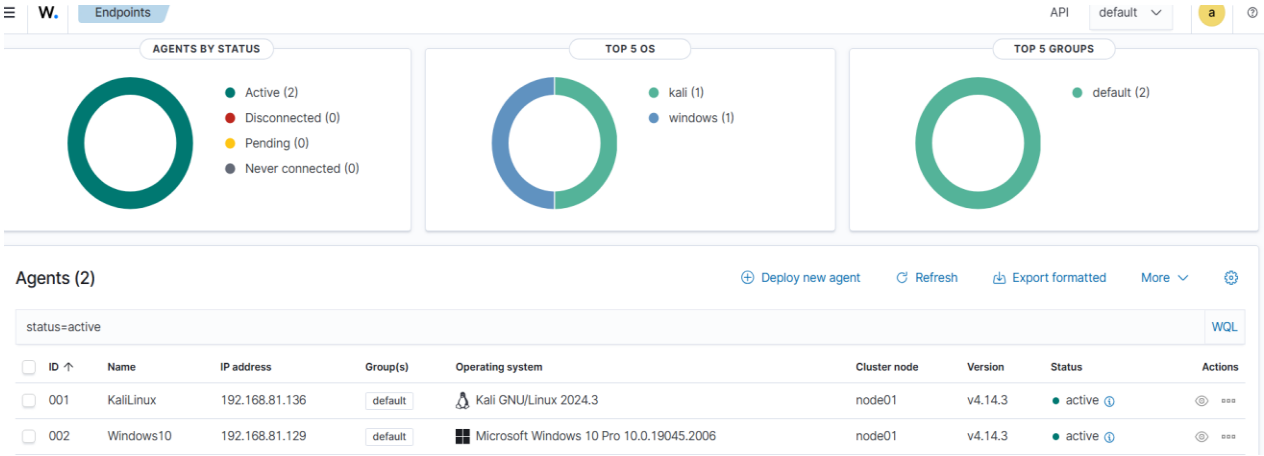
I ran the PowerShell command generated by the deployment wizard in an elevated PowerShell session. The MSI installer ran silently with the Manager IP passed as a parameter.

- Package: wazuh-agent-4.14.3-1.msi
- Manager IP: 192.168.81.135
- Agent name: Windows10
- Service started with: NET START WazuhSvc

Problem encountered: The first attempt to download the MSI failed with 'The remote name could not be resolved: packages.wazuh.com'. The Windows VM had no internet access. The second network adapter was incorrectly set to LAN Segment. After changing it to NAT and running ipconfig /release and ipconfig /renew, internet access was restored and installation succeeded.

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6
PS C:\Windows\system32> ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=59ms TTL=128
Reply from 8.8.8.8: bytes=32 time=61ms TTL=128
Reply from 8.8.8.8: bytes=32 time=81ms TTL=128
Reply from 8.8.8.8: bytes=32 time=81ms TTL=128

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 59ms, Maximum = 81ms, Average = 70ms
PS C:\Windows\system32> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.3-1.msi -OutFile $env:tmp\wazuh-agent.msi; msiexec.exe /i $env:tmp\wazuh-agent.msi /q WAZUH_MANAGER='192.168.81.135' WAZUH_AGENT_NAME='Windo
ws10'
PS C:\Windows\system32> NET START WazuhSvc

The Wazuh service was started successfully.

PS C:\Windows\system32> Get-Service WazuhSvc

Status   Name               DisplayName
------   ----               -----------
Running  WazuhSvc           Wazuh


PS C:\Windows\system32>
```

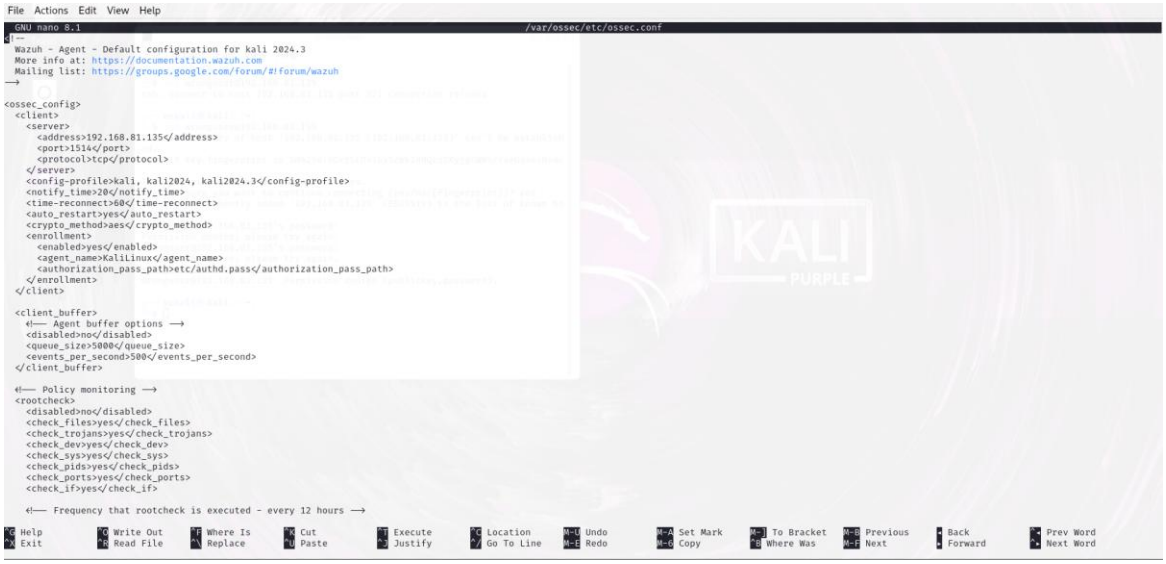
5.4 Both Agents Active in Dashboard

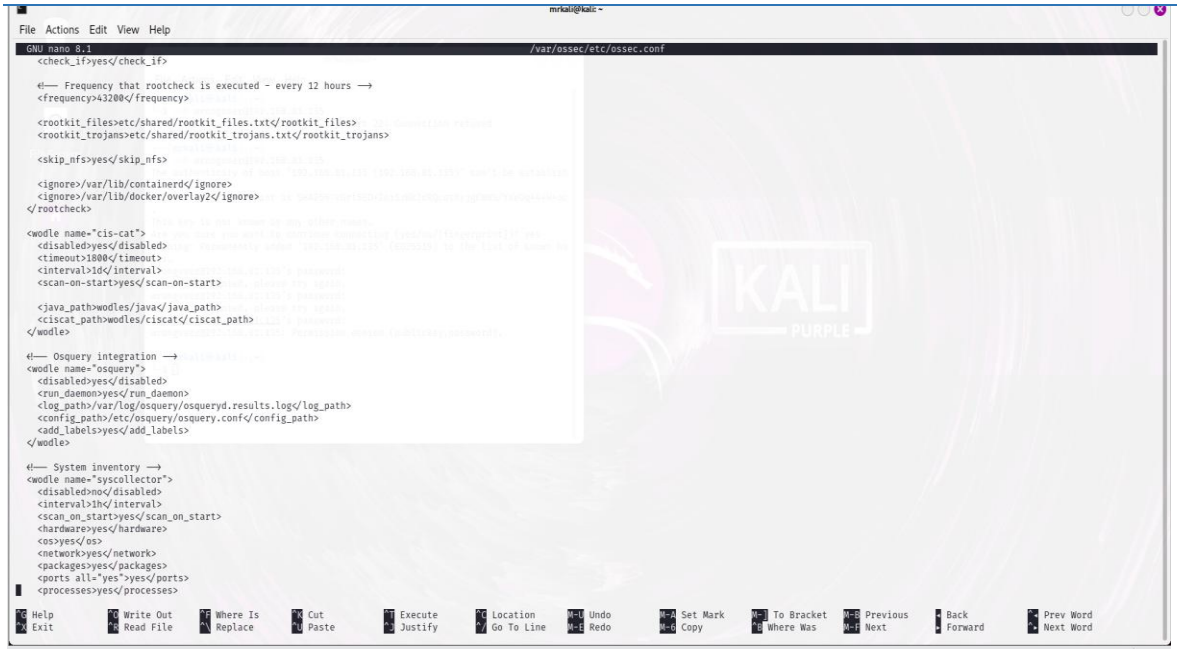


5.5 ossec.conf Review

I opened the ossec.conf configuration file on the Kali agent at /var/ossec/etc/ossec.conf and reviewed the three key sections:

- Server block: Defines the Wazuh Manager address (192.168.81.135) and port (1514/tcp). This is where the agent sends all collected data.
- Syscheck block: Controls File Integrity Monitoring. Monitors directories like /etc, /usr/bin, and /usr/sbin. Checks every 43200 seconds (12 hours) by default.
- Localfile blocks: Define which log files the agent reads from the operating system. On Kali this includes /var/log/auth.log (authentication events) and /var/log/syslog.





5.6 Pipeline Verification — Test Alerts

To confirm the full pipeline was working, I generated failed SSH login attempts from Kali to Ubuntu intentionally. I ran a loop to attempt 10 SSH connections with a non-existent username. After approximately two minutes, I checked the Wazuh dashboard for alerts.

- Command run: for i in {1..10}; do ssh -o ConnectTimeout=3 wronguser@192.168.81.135; done
- Each attempt was rejected with: Permission denied (publickey,password)
- Result: Alerts visible in dashboard within 2 minutes

Mar 6, 2026 @ 18:18:56.381 - Mar 7, 2026 @ 18:18:56.381

Export Formatted | Reset view | 694 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Mar 7, 2026 @ 18:14:37.062	Windows10	Software protection service scheduled successfully.	3	60642
Mar 7, 2026 @ 18:13:59.632	Windows10	Non service account logged off.	3	67023
Mar 7, 2026 @ 18:13:59.615	Windows10	Non service account logged off.	3	67023
Mar 7, 2026 @ 18:13:59.601	Windows10	Special privileges assigned to new logon.	3	67028
Mar 7, 2026 @ 18:13:59.592	Windows10	Windows Workstation Logon Success	3	60118
Mar 7, 2026 @ 18:13:59.586	Windows10	Windows Workstation Logon Success	3	60118
Mar 7, 2026 @ 18:13:57.628	Windows10	License activation (slui.exe) failed.	5	60646
Mar 7, 2026 @ 18:12:54.437	KaliLinux	PAM: Login session closed.	3	5502
Mar 7, 2026 @ 18:12:54.433	KaliLinux	PAM: Login session closed.	3	5502
Mar 7, 2026 @ 18:05:12.460	Windows10	Wazuh agent disconnected.	3	504
Mar 7, 2026 @ 17:21:59.211	Windows10	Software protection service scheduled successfully.	3	60642
Mar 7, 2026 @ 17:21:51.443	Windows10	License activation (slui.exe) failed.	5	60646
Mar 7, 2026 @ 17:21:50.843	Windows10	Name resolution for the name wpad timed out	5	61109
Mar 7, 2026 @ 17:21:39.544	KaliLinux	PAM: Login session opened.	3	5501
Mar 7, 2026 @ 17:21:39.485	KaliLinux	PAM: Login session opened.	3	5501

Rule Auntication failure:

5.7 What I Understood

The client-server model means the Manager never reaches out to endpoints. The agents initiate the connection and push data to the Manager. This matters architecturally because it means the Manager does not need network access to every endpoint — the endpoints need access to the Manager. Understanding this makes

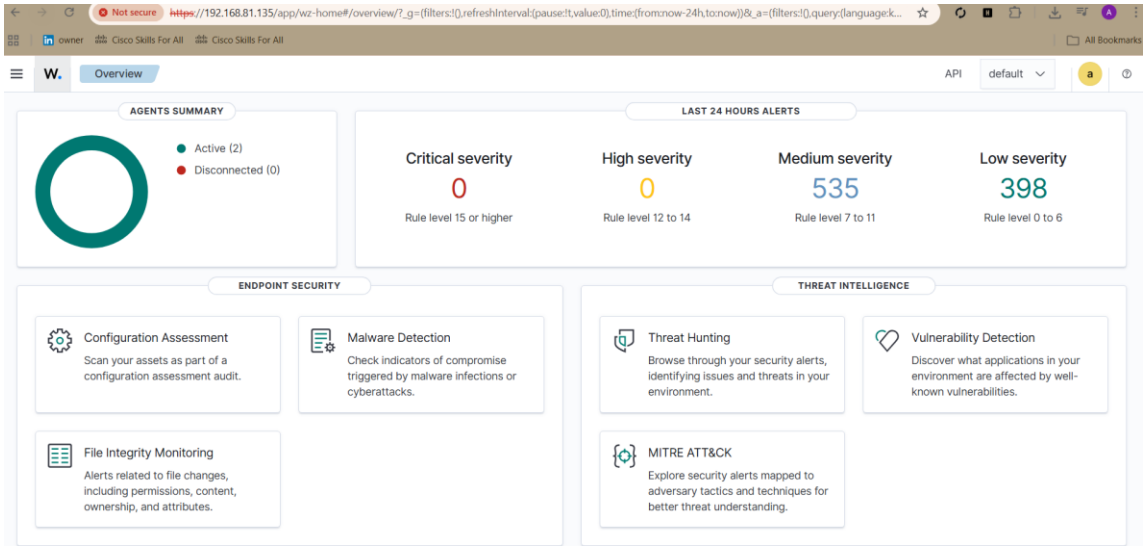
troubleshooting much faster: if an agent is not connecting, the problem is always on the agent side or the network path between agent and Manager.

6. Day 7 — Dashboard Navigation and Alert Triage

Day 7 marked the shift from setup work to analysis. I spent the day working through the Wazuh dashboard, learning to navigate its modules, and performing alert triage on the events my lab environment had generated.

6.1 Dashboard Modules Explored

- Overview: Summary of agents, last 24 hours alert counts by severity, and quick links to modules.
- Agents: Detailed status of each connected agent, OS, IP, version, and last keep-alive time.
- Threat Hunting: Query-based interface for searching and filtering all events. Supports DQL (Dashboard Query Language) for precise filtering.
- MITRE ATT&CK: Maps generated alerts to the MITRE ATT&CK framework techniques automatically.
- File Integrity Monitoring: Shows files that have been added, modified, or deleted on monitored endpoints.



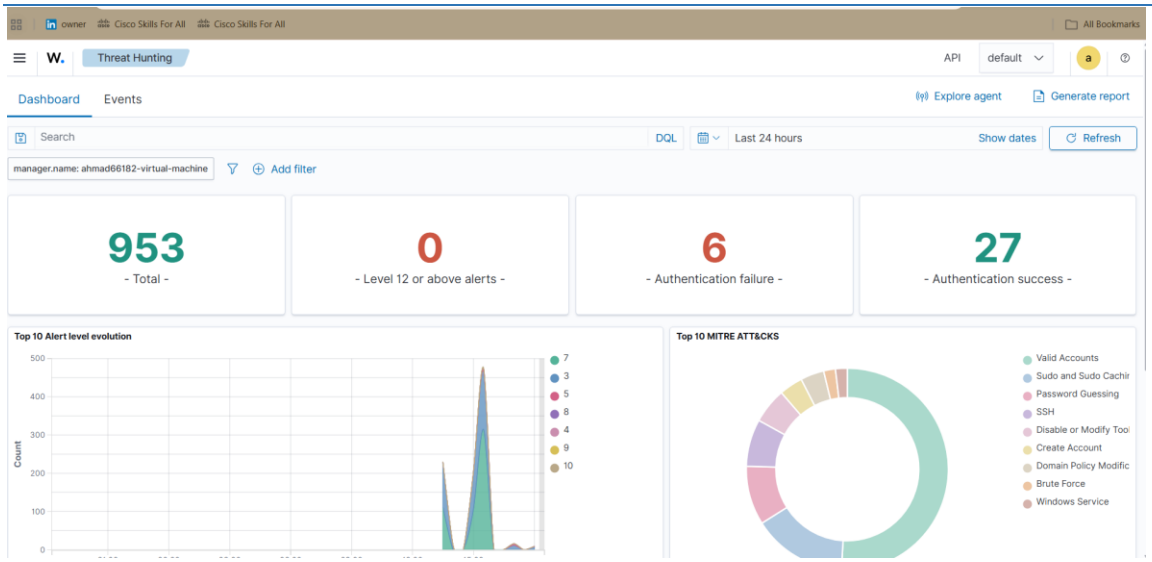


Figure THREAT HUNTING:

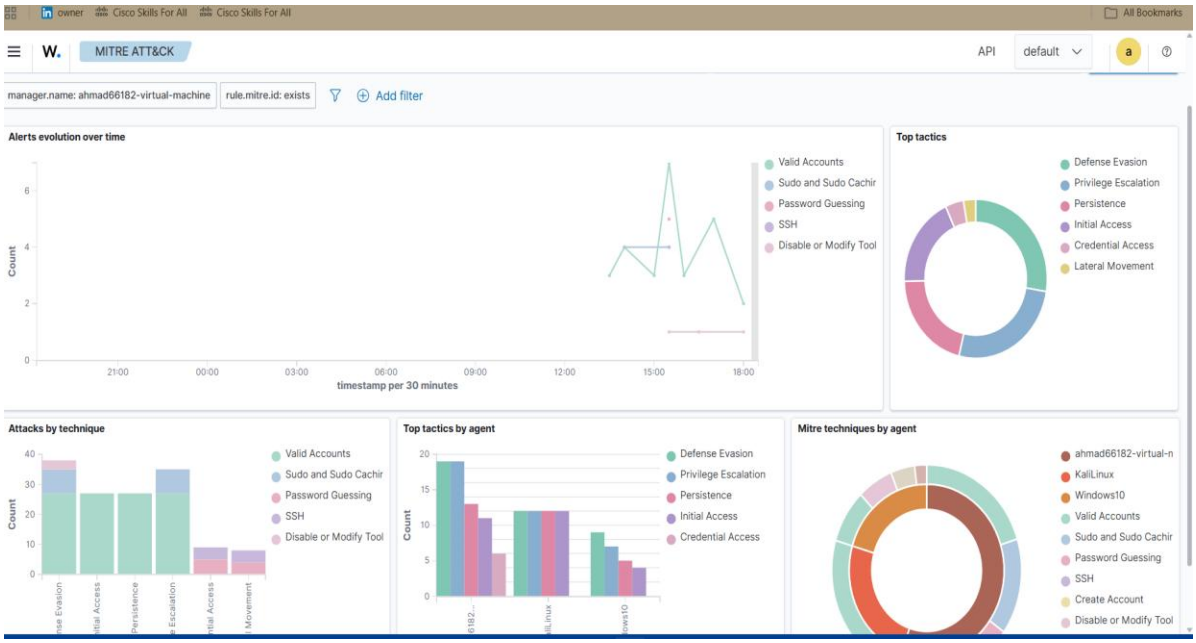


Figure MITRE ATTACK:

6.2 Wazuh Rule Levels

Wazuh assigns a numeric level to every alert to indicate severity. Understanding this scale is essential for prioritising triage work:

- Level 0–3: Informational — routine system activity, no action required
- Level 4–7: Low severity — worth noting, investigate if frequency is unusual
- Level 8–11: Medium severity — should be investigated
- Level 12–15: High and critical — immediate attention required

An important nuance: rule level is not the only factor. A Level 3 alert that fires 500 times in one hour is more significant than a single Level 8 alert triggered by a routine system task. Frequency and context must always be considered alongside severity.

6.3 Threat Hunting — Queries Practiced

- Filter by agent: agent.name: KaliLinux
- Filter by rule level: rule.level >= 7
- Filter by time range: Last 24 hours
- Filter by authentication events: rule.groups: authentication_failed

6.4 MITRE ATT&CK Mapping

The MITRE ATT&CK framework is a taxonomy of attacker techniques used across the security industry. Wazuh automatically maps alerts to ATT&CK techniques. In my lab environment, the following techniques were visible:

- T1110 — Brute Force / Password Guessing: Mapped from the failed SSH login attempts I generated from Kali.
- T1021 — Remote Services (SSH): Mapped from SSH connection events.
- T1078 — Valid Accounts: Mapped from successful authentication events.

Mapping to MITRE ATT&CK is useful because it provides a standard language for describing attacker behaviour that is consistent across tools and organisations. An alert described as 'SSH failure' is ambiguous — the same event mapped to T1110.001 (Brute Force: Password Guessing) is immediately understood by any security professional.

6.5 Alert Triage Report — Top 10 Alert Types

Rule ID	Level	Severity	What Triggered It	Assessment
5710	5	Low	SSH Authentication Failed — failed login attempt on Ubuntu server	Generated by Kali intentional SSH test. Frequency was high (10+ attempts). This is expected in the lab but in production it would indicate brute force — real concern at scale.
5715	3	Info	SSH Login Session Opened — successful SSH connection established	Normal activity when a legitimate user logs in. Not a concern unless from unknown IP.
554	3	Info	File added to system directory — new file detected by syscheck	Triggered when system updates add files. Routine in lab environment. Would investigate in production if unexpected.
550	7	Medium	File integrity check failed — monitored file was modified	Syscheck detected a change. Could indicate tampering. Would always investigate in a real environment.
5501	3	Info	User login session opened on endpoint	Normal Windows/Linux user authentication. Baseline activity.

Rule ID	Level	Severity	What Triggered It	Assessment
2502	3	Info	User missed login — incorrect credentials entered	Low frequency — likely a typo. High frequency would indicate brute force attempt.
5104	3	Info	Process execution detected on monitored endpoint	Normal system processes starting. Would flag if process name is unusual or unknown.
510	3	Info	Host-based anomaly detected — behaviour outside baseline	Context dependent. Single occurrence is routine. Needs correlation with other alerts.
31101	5	Low	Web attack signature detected in HTTP request	No web server in our lab — likely a false positive. In production, always investigate.
1002	2	Info	Unknown problem in log file — unrecognised log entry format	Usually benign. Wazuh encountered a log line it could not parse. Not a direct threat.

6.6 What I Understood

Alert triage is not about investigating every alert. It is about quickly distinguishing which alerts represent genuine threats, which are false positives, and which are normal baseline activity. In my lab environment, most alerts were informational events from normal system operation. The alerts that stood out were the SSH authentication failures I generated deliberately — which is exactly what a SOC analyst would look for in a production environment.

The key insight from Day 7 is that understanding your baseline is what enables threat detection. You cannot identify anomalous behaviour unless you know what normal looks like. Building that picture starts on Day 7 and develops over weeks.

7. Week 1 Reflections

7.1 Key Technical Learnings

- Network adapter configuration is the most failure-prone part of VM setup. Always verify IPs inside the VM, not just in the hypervisor settings.
- Wazuh agents communicate outbound to the Manager on ports 1514 and 1515. If agents are not connecting, check these ports first.
- The Wazuh Manager and Agent cannot run on the same machine — they conflict. Attempting to install the agent on the Ubuntu server (which already ran the Manager) produced a clear conflict error.
- Static IPs on the SIEM server are not optional. Dynamic IPs will break agent connections after a DHCP lease renewal.

7.2 Problems Encountered and Resolved

- Kali black screen after apt upgrade: Resolved by reinstalling open-vm-tools-desktop and the display manager. The upgrade broke the VMware display driver.
- Windows 10 no internet: Resolved by changing Network Adapter 2 from LAN Segment to NAT in VMware settings.

- Agent installation on Ubuntu failed: Understood the error — wazuh-agent conflicts with wazuh-manager. Agent correctly installed on Kali and Windows instead.

7.3 What Comes Next

Phase One continues with custom Wazuh detection rules, Suricata network intrusion detection, malware analysis on the Windows endpoint, and incident response procedures. All of this builds on the lab constructed this week. The value of having a working, stable environment is clear — every future week depends on it.